

今日から使えるGitHub入門

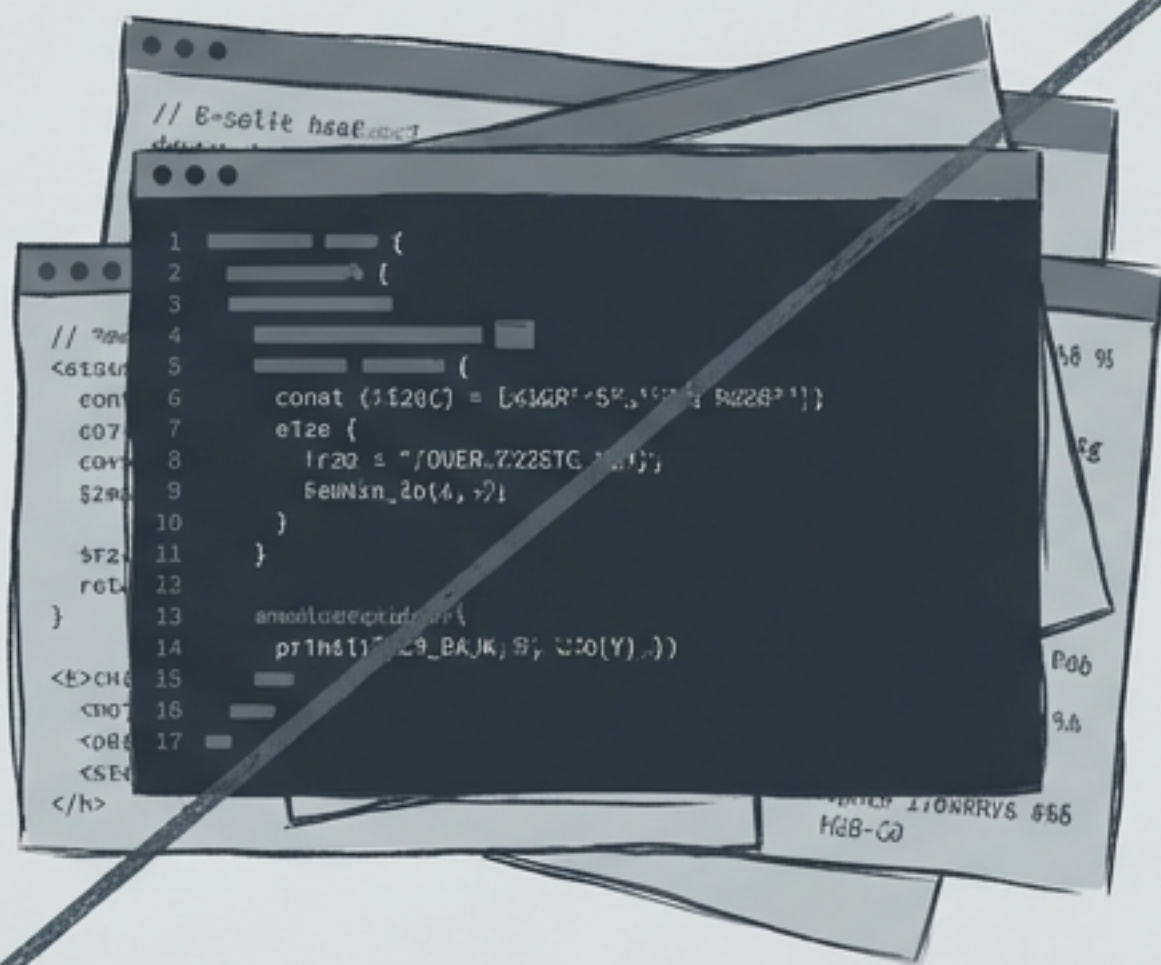
コード管理から業務改善まで



GitHubは単なるエンジニア向けの「ソースコード管理ツール」ではありません。

業務の履歴を残し、進捗を追い、承認プロセスを強固にするための「ビジネスOS」として機能します。

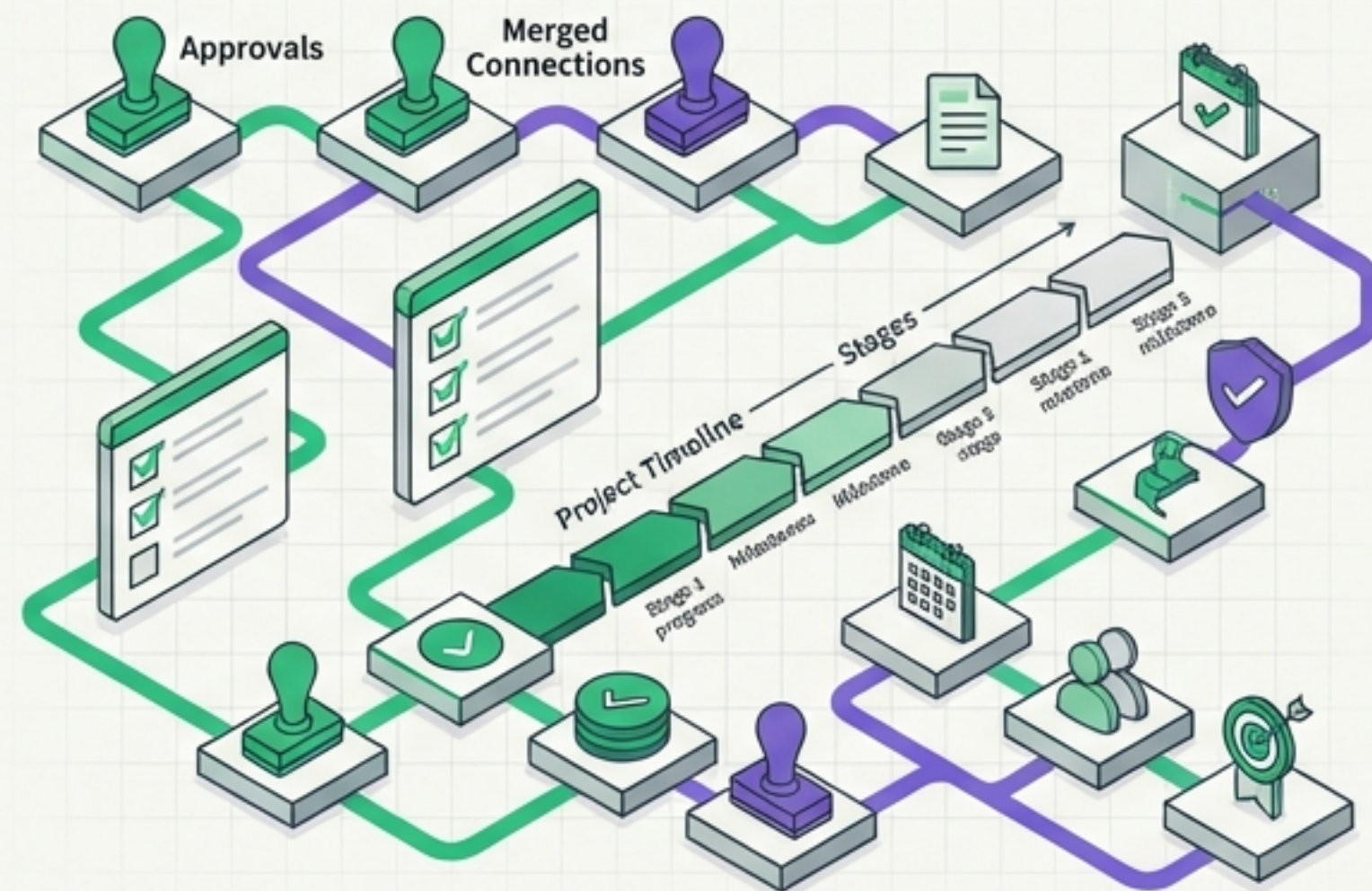
誤解



ソースコードの保管庫。

現実

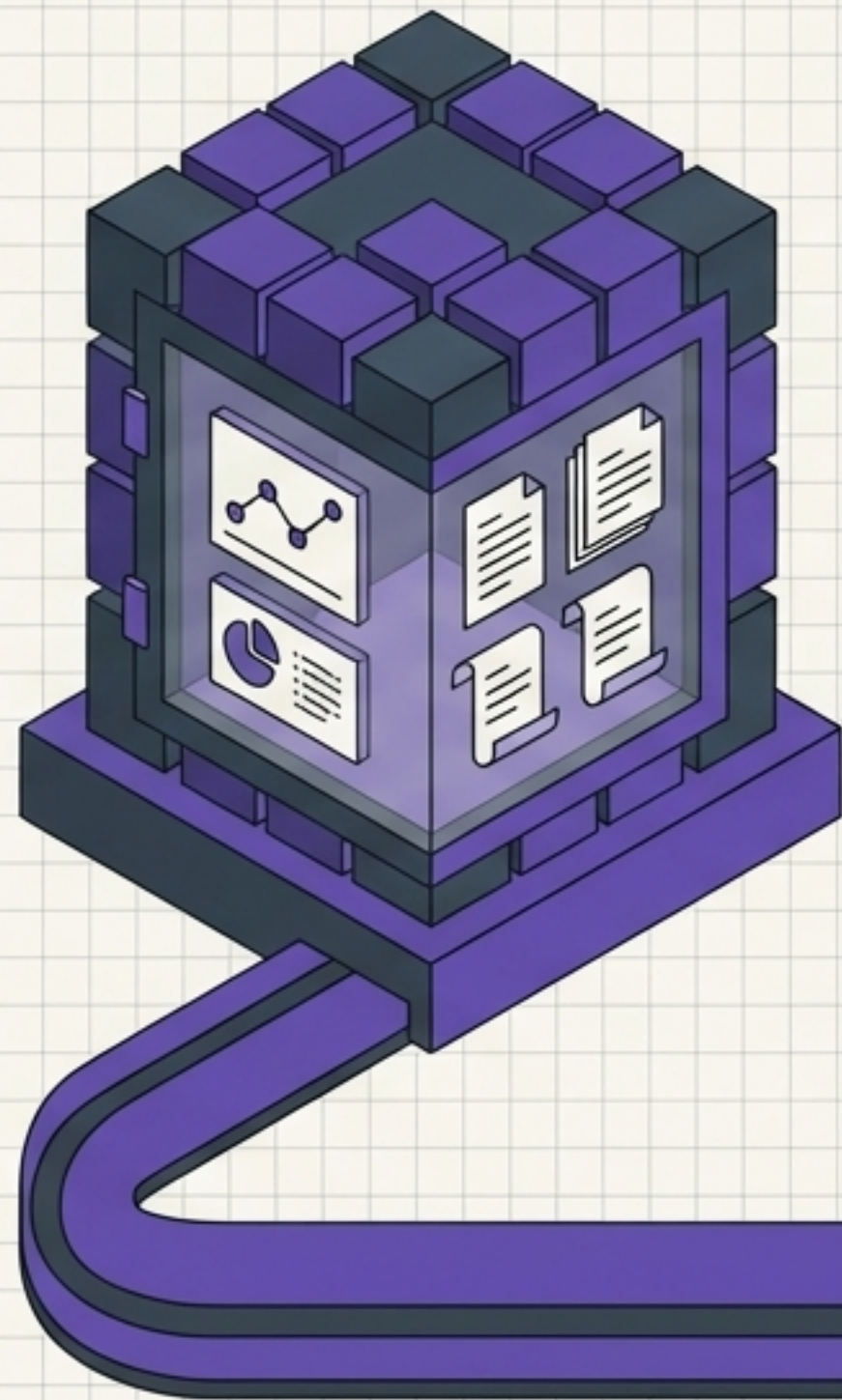
あらゆる業務の可視化エンジン。



従来の業務フローが抱える「情報のブラックボックス化」



あらゆるデータの保管と、改ざん不可能な「絶対的履歴」



Repository（保管場所）：ソースコードだけでなく、設計書、手順書、社内スクリプトなど、全作業の中心となるハブ。

Git（バージョン管理）：「いつ・誰が・何を変更したか」を永遠に記録。
過去の状態へ瞬時にロールバック可能。

2023.10.26
09:15 AM



初回コミット
(Initial Commit)

2023.10.26
14:30 PM



機能追加とマージ
(Feature Added & Merged)

2023.10.27
11:00 AM



緊急ロールバック
(Emergency Rollback)

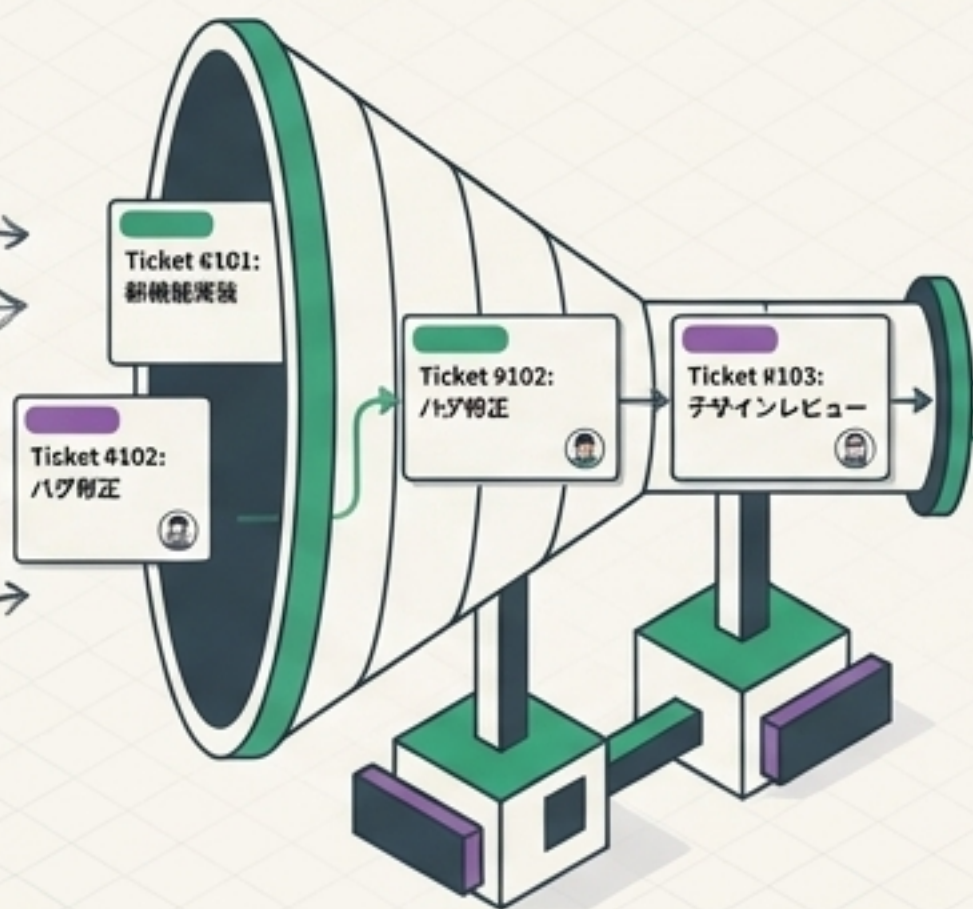
曖昧な依頼を「実行可能なタスク」へと変換する

Raw Requests

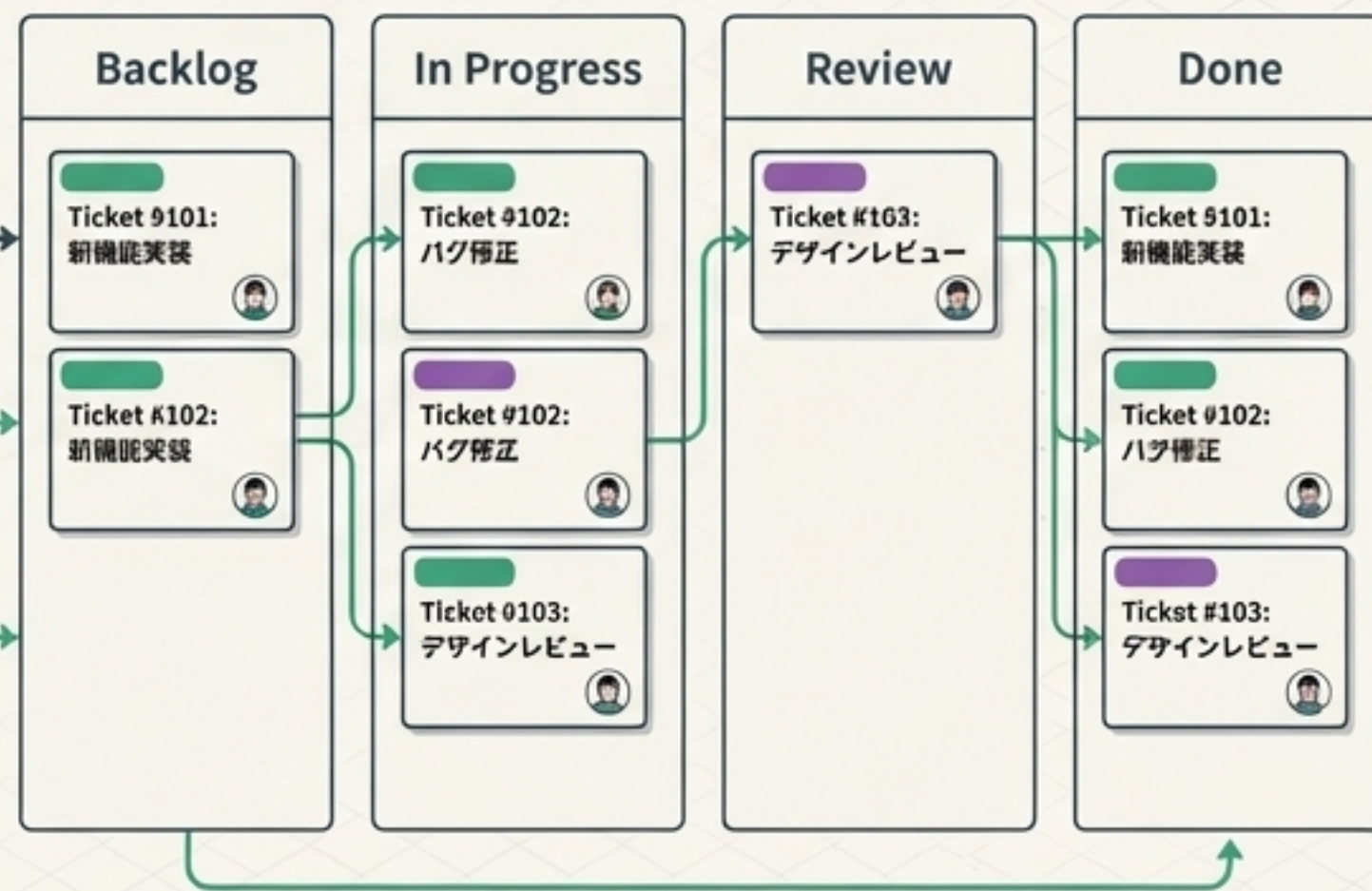


Issues (課題管理) :

「やるべき仕事」をチケット化。担当者設定とラベル分類で作業の抜け漏れを防止。



Projects (進捗可視化) : チケットをカンバン方式で管理。リアルタイムでチームの進捗とボトルネックを把握。



厳格な内部統制と監査対応を実現する「デジタル承認プロセス」



Pull Request（変更・承認管理）：

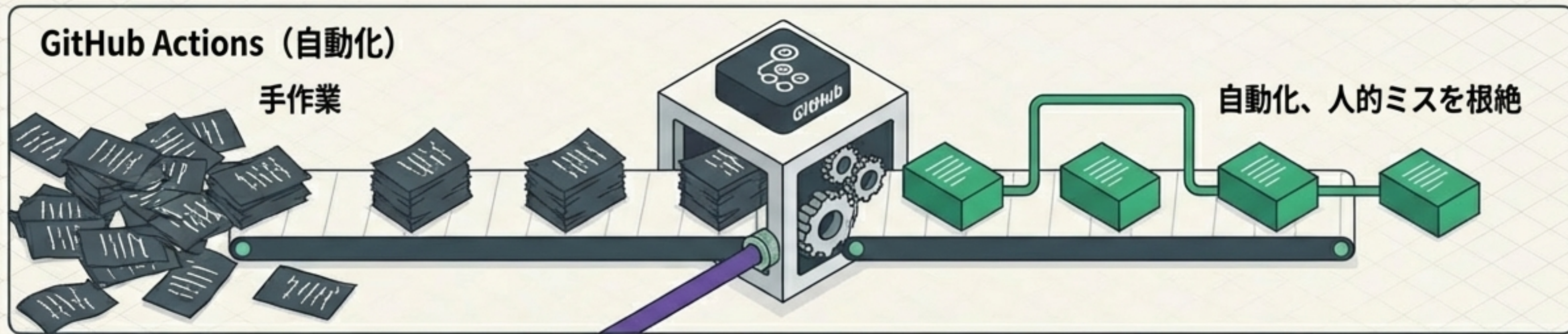
- ・ 個人判断での作業完了を排除。
- ・ 変更作業に対するレビューと承認プロセスを必須化。
- ・ 「誰が確認し、誰が承認したか」の証跡（監査ログ）が自動的に保存される。



Review & Comment



手作業を排除する「自動化」と、常に最新の「社内辞書」



GitHub Actions（自動化）：定期バッチ処理、テスト、デプロイなどの手作業を自動化し、人的ミスを根絶。

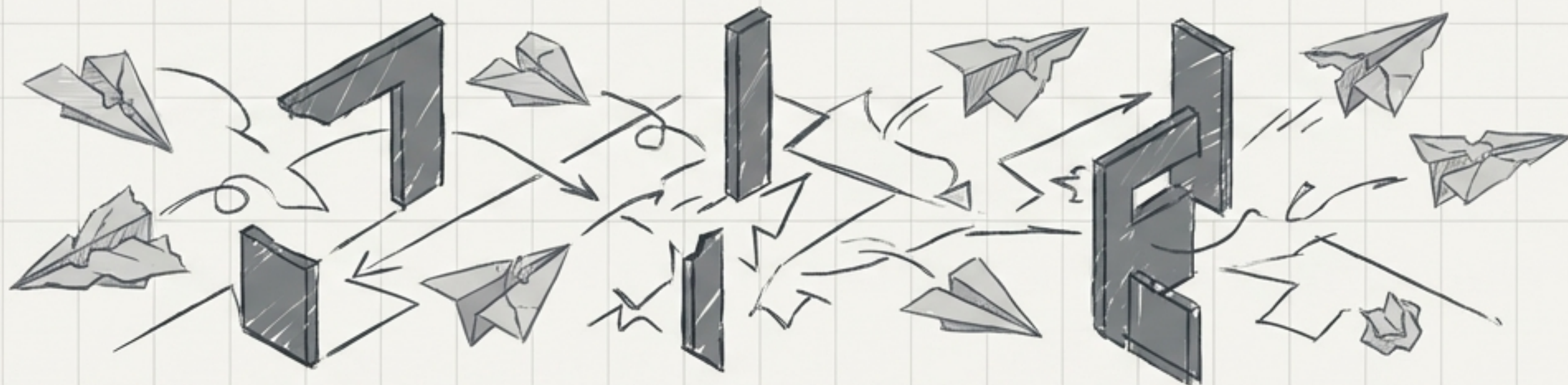


Wiki / Pages（社内辞書）：システム設計書や運用手順書を管理・公開。常に最新版のマニュアルに全員がアクセス可能。

コミュニケーションのサイロを破壊する一元管理

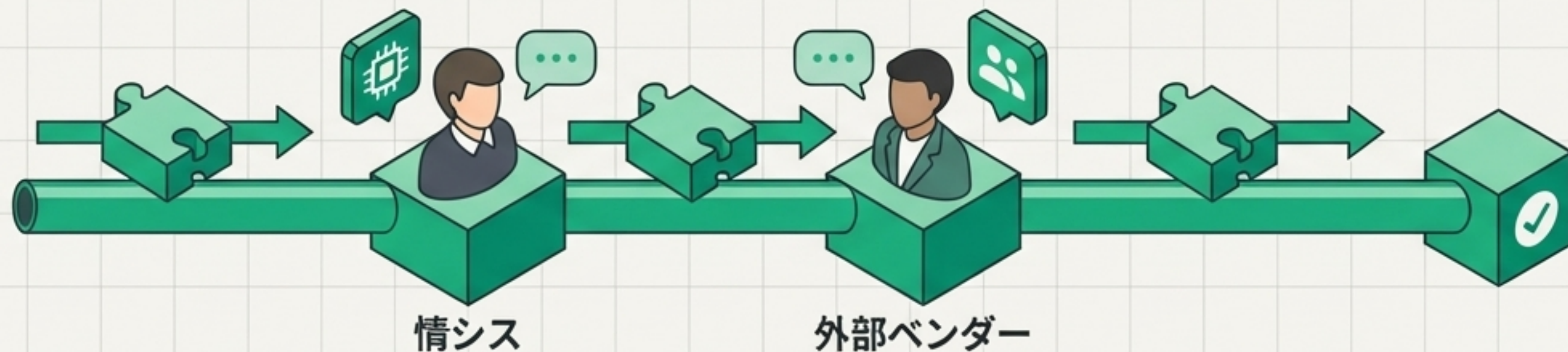
- 情シス問い合わせ: Issueとして登録することで、第三者からも対応状況が明確になり、対応漏れがゼロに。
- 外部ベンダー連携: 依頼内容と修正内容の記録を共有し、言った・言わないの認識ズレを防止。

Before



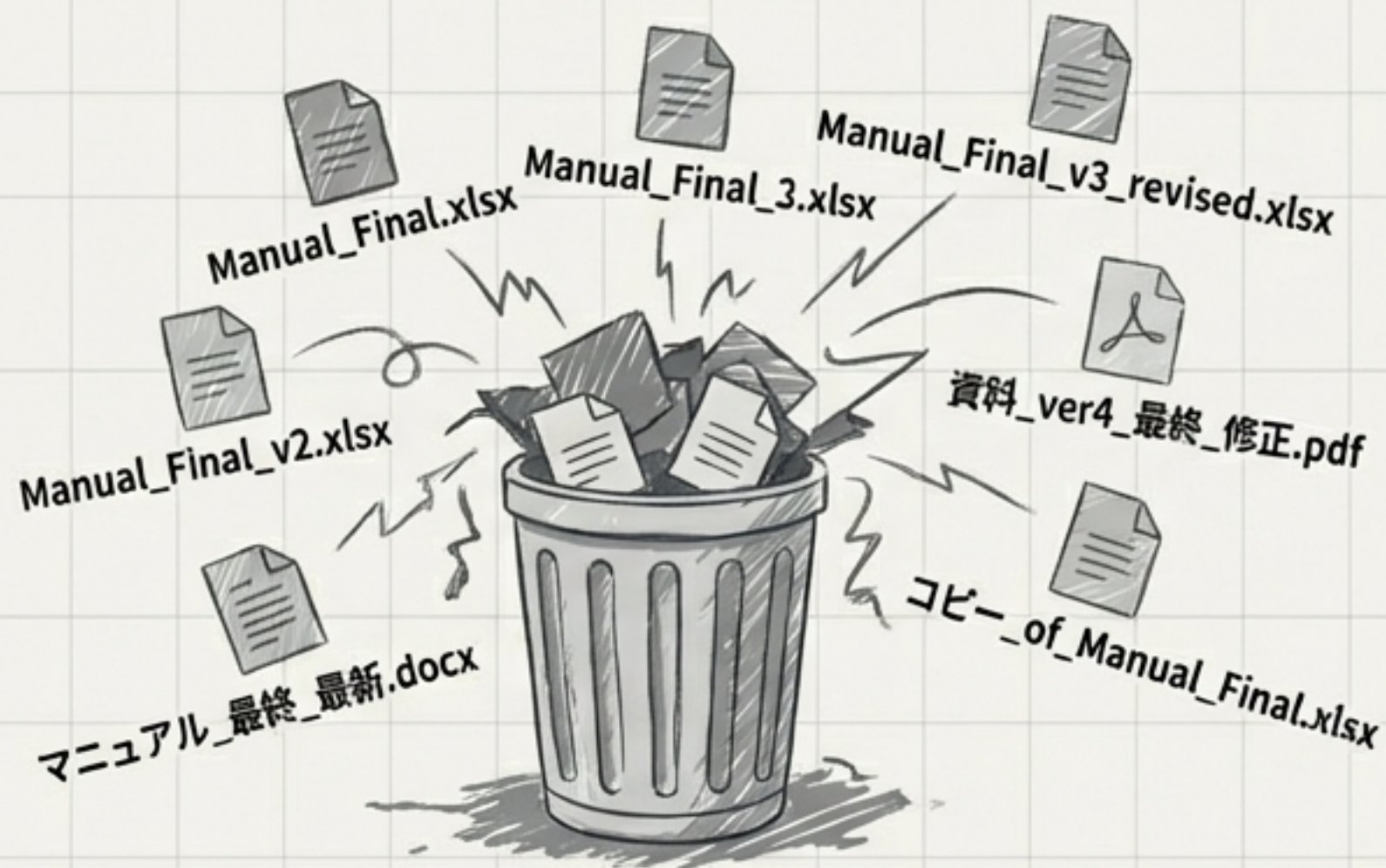
After

(GitHub Issues)



属人化の解消と「最終版ファイル迷子」からの脱却

Before: 「最終版」が不明なドキュメント管理



ドキュメント管理: 「どれが最新か分からない」状態を排除。ファイルの複製やローカル保存でバージョンが分散し、属人化が進行。

After: Gitによる「真の最新版」の一元管理

システム設計書.md

システム概要

1. 目的

- 属人化の解消、掲載した最新から分散しを修正
- 属人化の解消とによる重荷を修正

2. アーキテクチャ

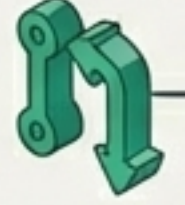
- マイクロサービス構成に、マイクロサービス構成修正
- マイクロサービスを修正

Current Version

- 2024-05-10 14:32:01
佐藤 健太
アーキテクチャ図を更新
- 2024-05-09 10:15:45
鈴木 美咲
アーキテクチャ図を更新
- 2024-05-09 10:15:45
鈴木 美咲
目的セクションを修正
- 2024-05-10 14:32:01
佐藤 健太
アーキテクチャ図を更新

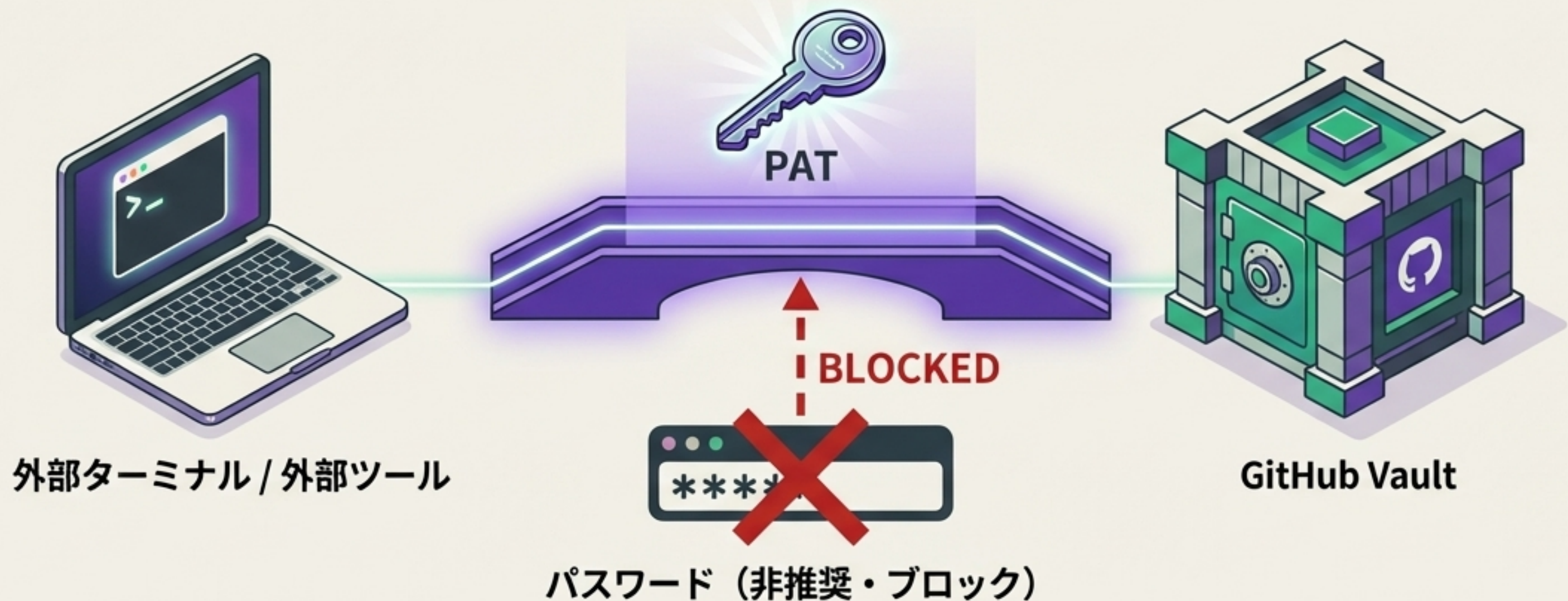
引き継ぎの自動化: MarkdownとGitで差分と更新履歴を自動保存。過去のやり取り自体が「業務経緯の履歴」となるため、専用の引き継ぎ資料作成が不要に。

GitHub導入による業務フロー変革マトリクス

従来の課題	GitHubでの解決策	もたらされる改善効果
口頭・メールでの依頼	 Issueで一元管理	抜け漏れ防止・履歴の蓄積
Excelによる進捗管理	 Projectsで可視化 	リアルタイム把握・会議効率化
作業履歴が残らない	 Commit履歴で自動記録	常に最新版が明確に
承認履歴が不明確	 Pull Requestで管理 	内部統制・監査対応の強化
属人化した業務	 履歴による共有 	引き継ぎ資料作成の負担軽減

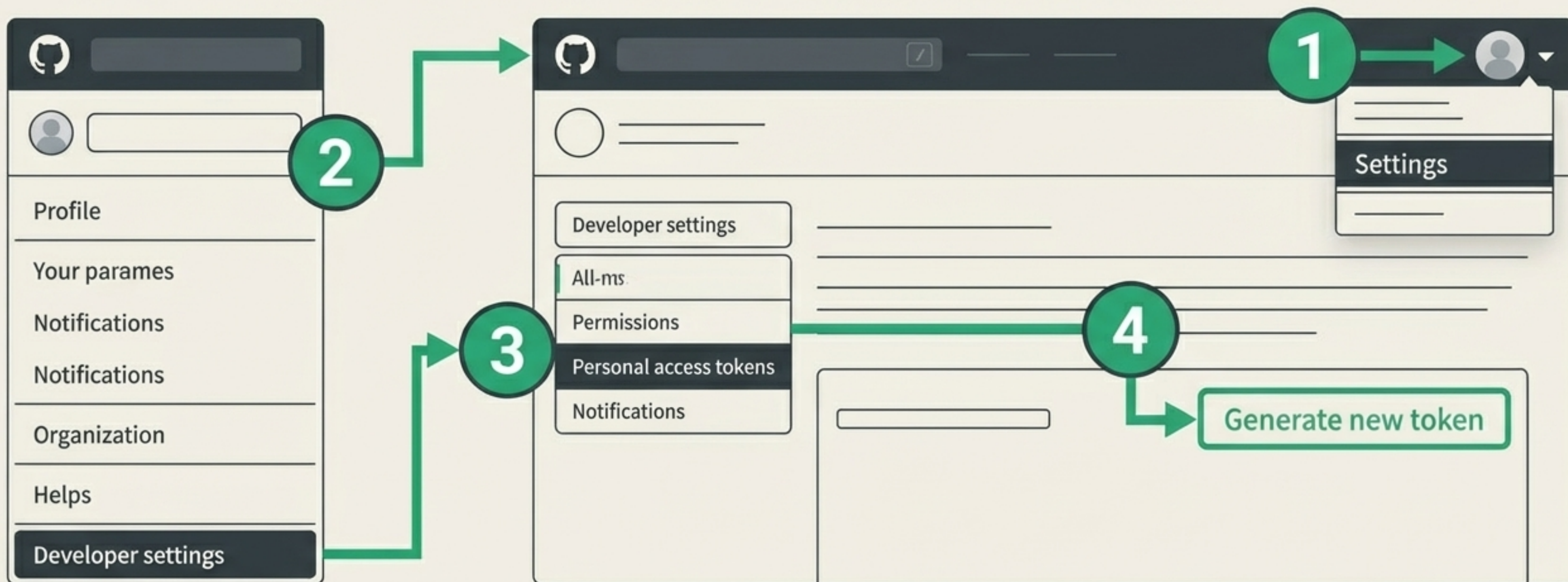
外部連携への第一歩：Personal Access Token（PAT）の役割

- GitHubの強力な機能をコマンドラインや外部ツールから安全に操作するための「専用の認証鍵」。
- パスワードの代わりとして機能し、セキュアな業務環境の構築に不可欠な初期設定です。

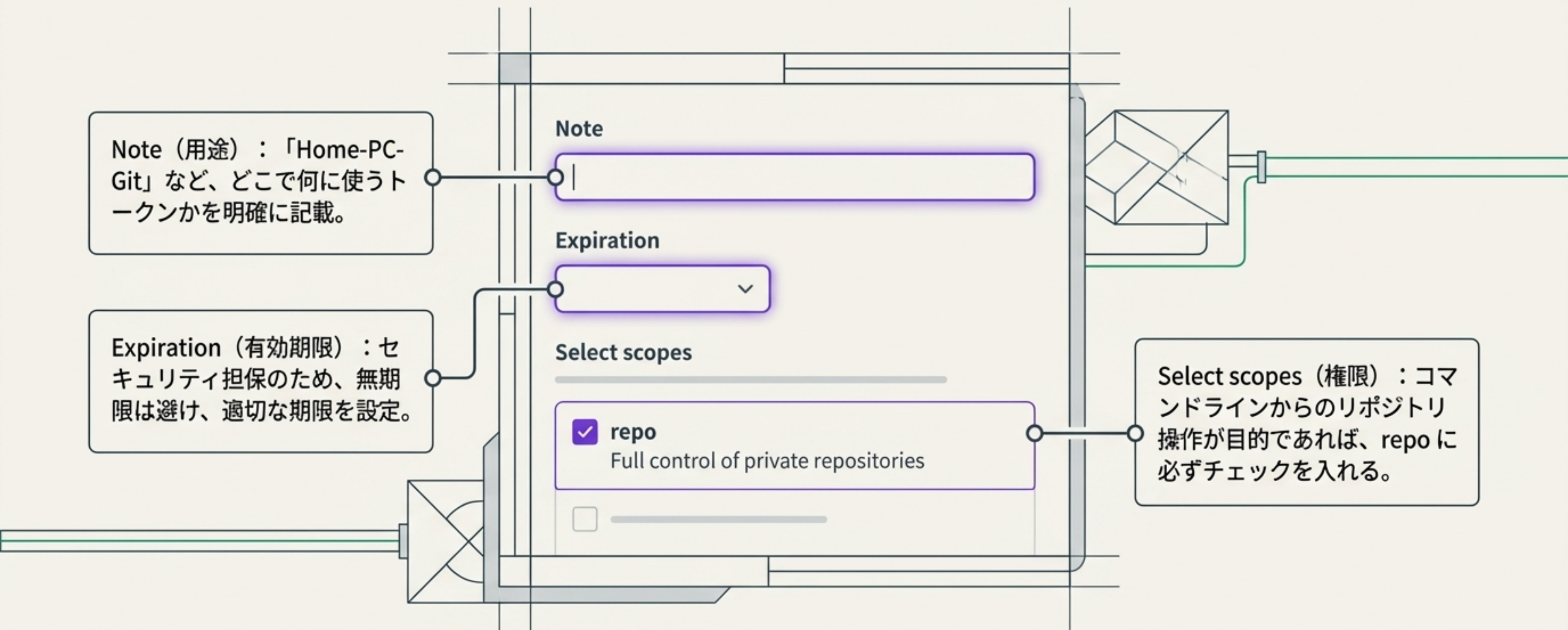


トークン発行手順：設定画面へのナビゲーション

- ・汎用的な連携が必要な場合は「Tokens (classic)」を選択。
- ・細かく権限を制限したい場合は「Fine-grained tokens」を選択。

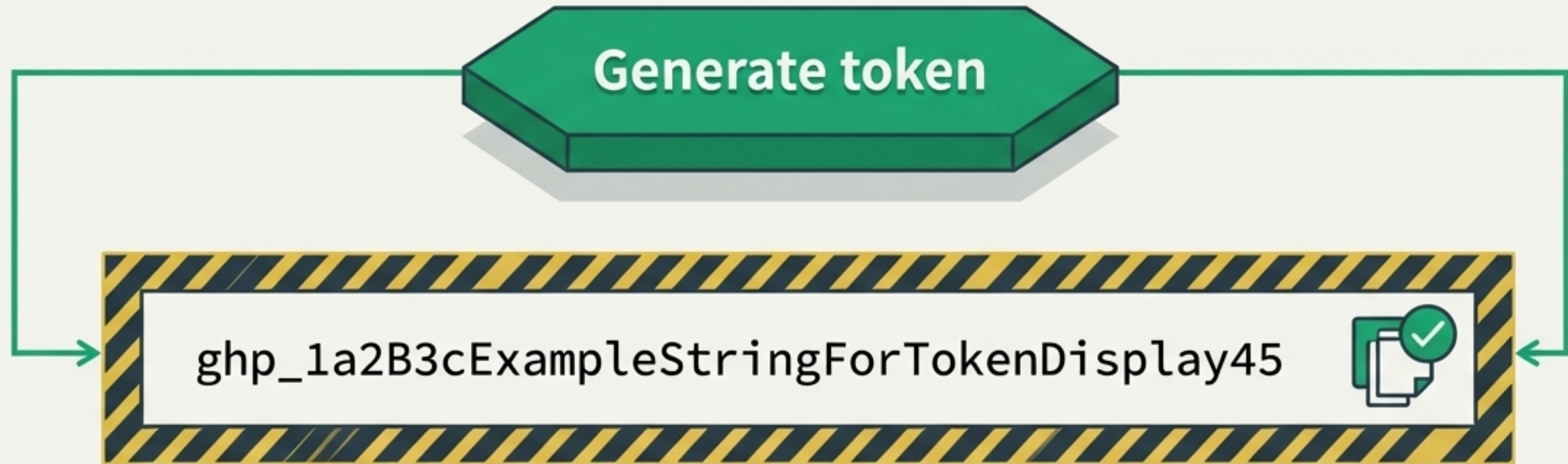


トークン発行手順：権限と期限の最適化



最終ステップ：一度きりのトークン表示と厳重な保管

- 生成されたトークンは、この画面を閉じると二度と表示されません。
- 必ずその場でコピーし、安全なパスワード管理ツール等に保存してください。
- ※ソースコード内や公開場所に絶対に貼り付けないこと。



「組織の記憶」が、次のアクションを加速する

- GitHubはツールではなく、組織のコミュニケーションと履歴を統合するインフラです。
- 分散して見えなかった業務から、透明性が高く、自動化された「次世代のワークスペース」へ。
- 今日発行したトークンが、その変革の鍵となります。

